

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[CVS/FAQ](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[WinCVS](#) › [FrontPage](#) › [개발자코너](#) › [CVS/GuideLine](#) › [KLDPNetCVS-HOWTO](#) › CVS/FAQ
CVS 사용에 대한 FAQ 모음

Login:

Password:

[Join](#)

한글로 된 CVS 문서들이 여럿 나와 있지만 꼭 필요한 정보들이 여기저기 흩어져 있고 체계적이지 않은 경우가 있어, 처음 CVS를 접하는 분들이 비슷한 질문을 반복적으로 하게 되는 듯 합니다. 오픈소스 프로젝트 개발에 참여하고자 하는 분들에게 필요한 정보를 모아봤으면 합니다. CVS 사용하면서 이 간단한 걸 몰라서 참 고생했었지 싶은게 있으면 언제든지 추가해 주세요.

Every purchase has its price.

[페이지 이름 띄어쓰기](#)
[RecentChangesMacro](#)
[Doxygen/강좌02](#)
[qps](#)

마침 [KLDPconf](#) 주제 응모에도 비슷한 얘기가 나와서 생각난 김에 일단 시작해 봅니다.

질문만 적어주셔도 물론 좋습니다.

Contents▲

1 질문과 답

- [1.1 CVS를 빨리 배워서 써야하는데 어떡하죠?](#)
- [1.2 CVS가 뭐예요? / 어디에 쓰는 물건인가요?](#)
- [1.3 CVS를 실제로 개발 과정에 사용하는 프로젝트가 있나요?](#)
- [1.4 CVS를 쓰면 그래서 뭐가 좋은가요?](#)
- [1.5 RCS는 또 뭐죠? / RCS 하고는 뭐가 다른가요?](#)
- [1.6 SourceSafe와는 뭐가 어떻게 다른가요?](#)
- [1.7 CVS에서한 점이 마음에 들지 않습니다. 다른 프로그램은 없나요?](#)
- [1.8 Windows에서 쓸 수 있는 CVS 클라이언트가 있나요?](#)
- [1.9 Windows에서 쓸 수 있는 CVS 서버가 있나요?](#)
- [1.10 CVSROOT 환경 설정을 어떻게 해야하나요?](#)
- [1.11 Anonymous checkout 이 뭔가요? 어떻게 하죠?](#)
- [1.12 로그인을 하라는데 어떻게 하나요?](#)
- [1.13 저장소\(Repository\)의 디렉토리 구조를 알고 싶습니다.](#)
- [1.14 새 Repository는 어떻게 만드나요?](#)
- [1.15 새 프로젝트는 어떻게 만드나요?](#)
- [1.16 checkout은 어떻게 하나요?](#)
- [1.17 checkout한 이후에 Repository에서 바뀐 내용을 확인하고 싶습니다.](#)
- [1.18 내가 작업한 내용은 어떻게 알 수 있나요?](#)
- [1.19 지금까지 작업한 내용을 checkin하고 싶습니다.](#)
- [1.20 Conflict가 생겼습니다! 어떻게 해야하나요?](#)
- [1.21 소스에서 이 부분을 누가 만든 건지 좀 알아야겠습니다.](#)
- [1.22 파일을 추가하고 싶습니다.](#)
- [1.23 파일을 삭제하고 싶습니다.](#)
- [1.24 파일 이름을 바꾸고 싶습니다.](#)
- [1.25 디렉토리를 지우고 싶습니다.](#)
- [1.26 실수로 잘못된 소스를 checkin해 버렸습니다. 어떻게 고쳐야하나요?](#)
- [1.27 checkout 받은 소스를 이제 그만 지워버리고 싶어요.](#)
- [1.28 현재까지 작업 내역을 보고 싶습니다.](#)
- [1.29 revision이 뭔가요?](#)
- [1.30 tag가 뭔가요?](#)
 - [1.30.1 tag는 어떻게 붙이나요?](#)
- [1.31 branch는 또 뭔가요?](#)
 - [1.31.1 branch는 어떻게 만들죠?](#)
 - [1.31.2 merge는 어떻게 하는 겁니까?](#)
- [1.32 과거 특정 시점에 소스가 어땠는지 보고 싶습니다.](#)
- [1.33 CVS 서버에 어떤 module이 있는지 어떻게 알 수 있나요?](#)
- [1.34 CVSROOT/modules에 모듈 등록은 어떻게 하나요?](#)
- [1.35 CVSROOT/passwd에 사용자를 어떻게 추가하나요?](#)
- [1.36 익명 사용자는 어떻게 등록하나요?](#)
- [1.37 CVS가 /root 아래 파일을 찾으려 합니다! 이게 뭔가요?](#)
- [1.38 Repository를 다른 기계로 옮기고 싶습니다.](#)
- [1.39 CVS 서버에 있는 repository를 미러링할 수 없나요?](#)
- [1.40 module들을 공유할 수 있는 방법이 있나요?](#)
 - [1.40.1 간단한 방법](#)
- [1.41 Source file에 강제로 revision을 지정할 수 있나요?](#)
- [1.42 CVS도 VSS처럼 배타적 락\(Exclusive Lock\)을 걸어 놓고 사용할 수 있나요?](#)
- [1.43 Repository에 Attic 이란게 있던데 이건 뭔가요?](#)
- [1.44 Unicode file을 지원하지 못하는가요?](#)



kss@kldp.org

- [1.45 이미 Text로 업로드 된 파일을 Binary로 바꿀 수 있는가요?](#)
- [1.46 이미 Binary로 업로드 된 파일을 Text로 바꿀 수 있는가요?](#)
- [1.47 로그를 잘못 적었는데 고칠 수 있을까요?](#)
- [1.48 Tag에 comment를 붙이는것은 가능한가요](#)
- [1.49 Unknown command 라는 에러가 납니다.](#)
- [1.50 항상 최신버전의 소스를 유지하고 싶어요](#)
- [1.51 이전버전으로 되돌리고 싶어요\(Rollback\)](#)
- [1.52 CVS를 쓸까요 Subversion을 쓸까요?](#)
- [2 help 사용법](#)
- [3 용어](#)
- [4 문서를 만들면서](#)
- [5 관련 자료](#)

1 질문과 답 ¶

[\[edit\]](#)

1.1 CVS를 빨리 배워서 써야하는데 어떻게하죠? ¶

[\[edit\]](#)

1. 우선 [DocbookSgml/ CVS-KLDP](#) 를 보고 한번 그대로 따라해 보세요.
2. 아래 정리해둔 다른 한글로 된 문서들을 틈틈히 읽어 봅니다.
3. CVS를 꾸준히 써야 하거나 여러가지 정석적인 사용 방법을 익히고 싶다면 [CVS-BestPractices](#) 문서를 정독하시기를 권합니다.
4. 큰 오픈소스 프로젝트에서 많은 개발자들이 같이 작업할 때 소스 버전 컨트롤이 어떤 방식으로 운영되는지에 관심 있는 분들은 [The CVS Book](#)을 한번 보시기 바랍니다.

1.2 CVS가 뭐예요? / 어디에 쓰는 물건인가요? ¶

[\[edit\]](#)

CVS(Concurrent Versions System)는 프로그램 소스나 문서 파일 버전 관리를 쉽게 할 수 있도록 도와주는 프로그램입니다. 특히 여러 사람들이 동시에 작업을 진행해야하는 규모가 큰 개발 프로젝트에서 큰 효과를 기대할 수 있습니다. 이밖에도 꾸준히 업데이트되는 문서 관리라든가 웹사이트 관리 등에 유용하게 쓸 수 있고, 심지어 /etc 아래에 들어가는 설정 파일을 (원격) 백업하는 용도로도 사용 가능합니다. ☺

동시에 여러 유닉스 장비에서 작업하는 경우 등에 써볼만한 "홈디렉토리 몽땅 CVS에 넣어버리기"에 대한 [LinuxJournal](#) 기사([CVS homedir](#))도 한번 읽어 보세요.

1.3 CVS를 실제로 개발 과정에 사용하는 프로젝트가 있나요? ¶

[\[edit\]](#)

잘 알려진 유명한 오픈소스 프로젝트들은 대개 CVS나 이와 유사한 소스 관리 시스템을 갖추고 있습니다.

1. [OpenBSD](#) : OS 및 시스템 전체 소스를 CVS로 관리하고 있습니다.
2. [OpenOffice](#) : 총 3만 파일에 대략 9백만 라인 이상되는 C++ 코드를 CVS로 관리합니다.
3. Mozilla : 전체 프로젝트 및 파생 프로젝트 소스 코드 및 릴리스/브랜치 관리를 CVS로 하고 있습니다.
4. [XFree86](#) : CVS 를 사용하는 유명 프로젝트중 하나입니다.
5. Apache : 위와 동일합니다.

1.4 CVS를 쓰면 그래서 뭐가 좋은가요? ¶

[\[edit\]](#)

관리자는 :

1. 프로젝트 참여자 개개인의 작업을 방해하지 않으면서 전체적인 진행 상황을 조절할 수 있다.
2. 릴리스된 버전과 개발중인 버전을 뒤섞지 않고 분리해서 관리할 수 있다.
3. 일정 기간에 진행된 작업 내용에 대한 리포트를 쉽게 만들어 낼 수 있다.
4. 일정 기간에 진행된 변경 내용을 효과적으로 추적할 수 있다.
5. 누가 일을 하고 누가 놀고 있는지 쉽게 감시할 수 있다. ☺

개발자는 :

1. 소스 백업에 신경 쓸 필요가 없다.
2. 새로운 코드를 작성할 때 기존 작업을 망칠 위험 없이 쉽게 시도할 수 있다.
3. 문제가 생겼을 때 어느 지점에서 문제가 생겼는지 쉽게 추적해서 찾아낼 수 있다.
4. 소스나 문서 특정 부분 작업을 누가 언제 했는지 쉽게 알아낼 수 있다.
5. 별도 리포트 작성 및 문서를 써야하는 건수나 양이 많이 줄어든다.
6. 맘먹으면 별로 한 일이 없어도 꾸준히 일하고 있는 것처럼 속일 수도 있다. ☺

1.5 RCS는 또 뭐죠? / RCS 하고는 뭐가 다른가요? ¶

[\[edit\]](#)

CVS는 RCS를 기반으로 작성된 버전 관리 시스템입니다. 쉽게 얘기하자면 RCS는 파일 하나하나에 대해 버전 관리를 해주는 것이고, CVS는 RCS 기능을 이용해 소스나 문서를 프로젝트/모듈 단위로 관리할 수 있도록 확장한 프로그램입니다. 이미 RCS를 쓰고 있고 익숙한 분이 아니라면 굳이 CVS를 새로 배울 필요는 없습니다. RCS로 할 수 있는 모든 것은 CVS로 할 수 있지만 그 역은 성립하지 않기 때문입니다.

CVS에서도 일부 기능(이미 써 놓은 로그 수정 등)은 RCS 명령을 사용해야 합니다. 궁금하다면

```
$ cvs -H admin
```

해서 RCS 관련 명령어를 한번 살펴 보세요.

1.6 SourceSafe와는 뭐가 어떻게 다른가요? ¶

[\[edit\]](#)

(<http://bbs.kldp.org/viewtopic.php?t=26590>)

1.7 CVS에서한 점이 마음에 들지 않습니다. 다른 프로그램은 없나요? ¶

[\[edit\]](#)

대체할 만한 것들...

- [Subversion](#)
- [arch](#)
- [OpenCM](#)
- [Stellation](#)
- [PRCS](#)
- [aegis](#)
- [eclipse](#)
- [Meta-CVS](#)
- [CVSup](#)
- lcvs

Non-Free

- [BitKeeper](#)
- [Perforce](#)
- [ClearCase](#)
- [SourceSafe](#)

1.8 Windows에서 쓸 수 있는 CVS 클라이언트가 있나요? ¶

[\[edit\]](#)

- [WinCVS](#)
 - [WinCVS 사용법 한글 설명](#)
- [TortoiseCVS](#)
 - [TortoiseCVS 사용법 한글 설명](#)
- [Eclipse](#)
- [VC++ 6 IDE와 통합된 CVS tool](#)
- [Visual Studio를 비롯하여 여러 IDE들과 통합된 CVS tool](#)

등이 있습니다.

CVS를 처음 사용하는 분에게는 [TortoiseCVS](#) 를 권합니다. Java를 주로 사용하거나 좋은 IDE 환경을 원하시는 분은 Eclipse를 한번 써보세요.

[DeleteMe](#) 윈도우즈 환경에서 CVS를 사용하는 방법에 대해 아직 쉽게 참고할 만한 문서가 없는 것으로 압니다. 누군가 위에 제가 링크해 드린 두 개의 내용을 바탕으로 해서 윈도우즈 환경에서 CVS를 사용하는 방법을 다른 별도의 문서를 만들어 주시면 참 좋겠네요. -- [권순선](#)
[WinCVS](#) 페이지에서 새롭게 문서를 작성해 나갈까 합니다. 많은 관심과 도움 부탁드립니다. -- [쫘아](#)

1.9 Windows에서 쓸 수 있는 CVS 서버가 있나요? ¶

[\[edit\]](#)

- [CVSNT](#)
- [CVSNT 설치하기 영문 설명서](#)
- [CVSNT 사용하기 한글 설명서](#)

1.10 CVSROOT 환경 설정을 어떻게 해야하나요? ¶

[\[edit\]](#)

Local Repository

```
export CVSROOT=/usr/local/cvsroot
```

Remote Repository

```
export CVSROOT=:pserver:username@host.domain.com:/var/cvsroot
```

Remote Repository w/ [SecureShell](#)

```
export CVS_RSH=ssh
export CVSROOT=:ext:username@host.domain.com:/var/cvsroot
```

1.11 Anonymous checkout 이 뭔가요? 어떻게 하죠? ¶

[\[edit\]](#)

익명 사용자에게 읽기 권한을 제공함으로써 소스나 문서 최신 버전을 손쉽게 업데이트할 수 있도록 해줍니다. 특히 컴파일이 필요없는 스크립트 기반 프로젝트에 유용합니다. [MoniWiki](#)를 예로 들자면

```
$ cvs -d:pserver:anonymous@cvs.kldp.net:/cvsroot/moniwiki login
$ cvs -z3 -d:pserver:anonymous@cvs.kldp.net:/cvsroot/moniwiki co moniwiki
```

와 같이 하면 됩니다. 이후에 새 버전이 나오거나 패치가 업데이트 되면

```
$ cvs update -dP
```

해서 간단히 최신버전으로 업데이트할 수 있습니다.

1.12 로그인을 하라는데 어떻게 하나요? ¶

[\[edit\]](#)

```
$ cvs login
(Logging in to user@host.domain.com)
CVS password:
```

```
$ cat ~/.cvspass
:pserver:user@host.domain.com:/var/cvsroot XYZ123
```

```
$ cvs logout
(Logging out of user@host.domain.com)
```

1.13 저장소(Repository)의 디렉토리 구조를 알고 싶습니다. ¶

[\[edit\]](#)

```
/home/cvs/ .... (1)
/home/cvs/CVSROOT/ .... (2)
/home/cvs/CVSROOT/passwd .... (3)
/home/cvs/CVSROOT/readers .... (4)
/home/cvs/module_A/ .... (5)
/home/cvs/module_B/ .... (6)
```

(1) \$CVSROOT 로 지정되는 CVS의 최상위 디렉토리

(2) CVS 저장소의 설정 파일들이 있는 디렉토리

(3) CVS 사용자 ID와 비밀번호

(4) CVS 읽기만 가능한 사용자

(5), (6) 프로젝트 디렉토리, 프로젝트에 따라 CVS 명령으로 생성됩니다.

1.14 새 Repository는 어떻게 만드나요? ¶

[\[edit\]](#)

```
$ export CVSROOT=/home/user/cvsroot
$ cvs init
$ ls /home/user/cvsroot
CVSROOT/
$ ls /home/user/cvsroot/CVSROOT/
Emptydir/      config      editinfo,v   modules,v    taginfo
checkoutlist   config,v    history      notify       taginfo,v
checkoutlist,v cvswrappers loginfo      notify,v     val-tags
commitinfo     cvswrappers,v loginfo,v    rcsinfo     verifymsg
commitinfo,v   editinfo    modules      rcsinfo,v   verifymsg,v
```

1.15 새 프로젝트는 어떻게 만드나요? ¶

[\[edit\]](#)

```
$ ls proj/
README a.c a.h
$ cd proj/
$ cvs import -m "this is the project" proj VENDOR INIT
$ ls /var/cvsroot/
CVSROOT/ proj/
$ ls /var/cvsroot/proj/
README,v a.c,v a.h,v
```

1.16 checkout은 어떻게 하나요? ¶

[\[edit\]](#)

```
$ cd ~/work
$ cvs co proj
U proj/README
U proj/a.c
U proj/a.h
```

특정 태그를 지정해서 받고 싶다면 다음과 같이 합니다.

```
$ cvs co -r RELEASE_1_0_5 moniwiki
```

1.17 checkout한 이후에 Repository에서 바뀐 내용을 확인하고 싶습니다. ¶

[\[edit\]](#)

우선 내용 확인 (로컬에 아무 변화도 만들지 않음).

```
$ cvs -n update
```

그리고 이상이 없으면 변경된 사항을 받아온다.

```
$ cvs update -dP
```

1.18 내가 작업한 내용은 어떻게 알 수 있나요? ¶

[\[edit\]](#)

```
$ cvs diff
$ cvs diff -r BASE
$ cvs diff -r BASE -r HEAD
$ cvs diff -c -r1.6 prog.c
$ cvs diff -c -r1.6 -r1.7 prog.c
```

1.19 지금까지 작업한 내용을 checkin하고 싶습니다. ¶

[\[edit\]](#)

```
$ cvs ci -m "fixed bug #12345"

$ cvs com[mit] prog.c
$ cvs ci -m "another bug fix in sheet.c" sheet.c
```

1.20 Conflict가 생겼습니다! 어떻게 해야하나요? ¶

[\[edit\]](#)

당황하지 마세요. 소스나 문서에서

```
<<<<
....
-----
....
>>>>
```

이렇게 생긴 부분을 찾아서 고친 다음 다시 cvs update; cvs ci 하면 됩니다.

1.21 소스에서 이 부분을 누가 만든 건지 좀 알아야겠습니다. ¶

[\[edit\]](#)

```
$ cvs annotate a.c
```

1.22 파일을 추가하고 싶습니다. ¶

[\[edit\]](#)

```
$ cvs add newfile
$ cvs add newdir
```

1.23 파일을 삭제하고 싶습니다. ¶

[\[edit\]](#)

```
$ cvs rm oldfile
$ cvs rm -f badfile
$ cvs rm -f olddir/badfiles
$ cvs rm olddir
```

1.24 파일 이름을 바꾸고 싶습니다. ¶

[\[edit\]](#)

단순히 파일이름만 바꾸는 방법과 history 까지 보존하는 방법이 있습니다 (매뉴얼에서 발취)

일반적인 방법

```
$ mv old new
$ cvs remove old
$ cvs add new
$ cvs commit -m "Rename old to new" old new
```

일반적인 만큼 방법도 쉽습니다. 기존 파일의 이름을 다른 이름으로 바꾼후 기존 파일을 cvs remove 로 제거하고

새로 바뀐 이름의 파일을 cvs add 로 추가합니다. 그 다음엔 remove와 add 만으로 파일이 지워지거나 추가된것이 적용되지 않으므로

이 두파일을 commit 해줘야 합니다

히스토리 까지 보존하는 방법

먼저 이 방법은 CVS repository 를 직접 접근하므로 조금은 위험하다고 매뉴얼에서 밝히고 있네요

```
$ cd $CVSROOT/module
$ mv old,v new,v
```

CVS repository 의 수정할 module 디렉토리에서 이름을 변경할 파일의 이름을 직접 바꾸어 주는 방법입니다

이것은 아래와 같은 장단점을 가지고 있습니다 **장점**

- change log가 그대로 보존됩니다
- revision 역시 그대로 유지됩니다

단점

- module 의 예전 release 나 tag 를 불러오기가 쉽지가 않다.
- 이름이 변경되었다는 정보가 log 에 남지 않는다
- 위의 작업(\$CVSROOT 에 들어가 파일을 직접 rename 하는..)을 하는 동안 누군가 cvs 작업을 한

다면 난처한 일이 생긴다.

1.25 디렉토리를 지우고 싶습니다 ¶

[\[edit\]](#)

CVS에서는 디렉토리 단위로 저장을 하기 때문에 히스토리를 유지하기 위해서는 그 디렉토리가 필요합니다. 따라서 진짜로 디렉토리를 지울 수는 없습니다. 대신 update 시에 받지 않는 방법이 있습니다. 먼저 **파일을 지우기**를 이용해서 해당 디렉토리의 파일을 모두 지웁니다. 그리고 나서 상위 디렉토리로 가서 해당 디렉토리를 지우고 나면 다음부터는 cvs update -P 로 업데이트 하면 해당 디렉토리가 보이지 않게 됩니다.

```
cd unneeddir
rm *           # 이때 CVS 디렉토리는 남겨져 있어야 합니다.
cvs remove
cvs commit    # 여기서 unneeddir의 모든 파일이 지워집니다
cd ..
cvs remove unneeddir
cvs update -P
```

또는 CVSRROOT에 가서 직접 디렉토리를 지워 버리는 방법이 있습니다. 대신이 방법은 CVSRROOT 디렉토리 관리 권한을 가지고 있어야 하고, 히스토리까지 사라진다는 단점이 있습니다.

1.26 실수로 잘못된 소스를 checkin해 버렸습니다. 어떻게 고쳐야하나요? ¶

[\[edit\]](#)

1.27 checkout 받은 소스를 이제 그만 지워버리고 싶어요. ¶

[\[edit\]](#)

```
$ cvs release -d proj
```

1.28 현재까지 작업 내역을 보고 싶습니다. ¶

[\[edit\]](#)

```
$ cvs st[atus] [-v] [prog.c]
$ cvs log [prog.c]
$ cvs ann[otate] [main.c]
```

1.29 revision이 뭔가요? ¶

[\[edit\]](#)

파일 하나하나에 붙어 있는 버전 번호를 말합니다.

```
$ cvs st README
.....
Working revision:    1.1
.....
```

1.30 tag가 뭔가요? ¶

[\[edit\]](#)

여러 파일이나 모듈에 버전을 표시하기 위해 한꺼번에 붙여둔 표시입니다.

```
$ cd moniwiki/
$ cvs st -v README
.....
Existing Tags:
    moniwiki-1_1          (branch: 1.1.2)
    RELEASE_1_0_5        (revision: 1.1)
.....
```

1.30.1 tag는 어떻게 붙이나요? ¶

[\[edit\]](#)

```
$ cvs tag TAGNAME
```

[[edit](#)]

- Vender Branch

```

+-----+
Branch 1.2.2.3.2 -> ! 1.2.2.3.2.1 !
/
/
+-----+ +-----+ +-----+ +-----+
Branch 1.2.2 -> !_! 1.2.2.1 !----! 1.2.2.2 !----! 1.2.2.3 !----! 1.2.2.4 !
/ +-----+ +-----+ +-----+ +-----+
/
+-----+ +-----+ +-----+ +-----+ +-----+
! 1.1 !----! 1.2 !----! 1.3 !----! 1.4 !----! 1.5 ! <- The main trunk
+-----+ +-----+ +-----+ +-----+ +-----+
!
!
+-----+ +-----+ +-----+
Branch 1.2.4 -> +---! 1.2.4.1 !----! 1.2.4.2 !----! 1.2.4.3 !
+-----+ +-----+ +-----+

```

[[edit](#)]

```
$ cvs rtag -b -r release-1 release-1-patches project
$ cvs checkout -r release-1-patches project
$ cvs commit -m "Fixed printf bug" foo.c
$ cd ..; cvs release -d project
```

이 명령을 실행한 작업소는 앞으로 이 브랜치로 작업을 하게 됩니다.

1.31.2 merge는 어떻게 하는 겁니까? ¶

[\[edit\]](#)

```
$ cvs checkout -j release-1-patches project
$ cvs commit -m "Merged patch"
```

1.32 과거 특정 시점에 소스가 어땠는지 보고 싶습니다. ¶

[\[edit\]](#)

태그를 붙여 두지 않았다면, 날짜와 시간을 써서 다음과 같이 작업하는 것도 가능합니다.

```
$ cvs rtag -b -D "2000-01-01 00:00" first-millennium-release project
$ cvs get -D "3 month ago" myproj
$ cvs get -D "1999-11-23 09:00" oldproj
```

CVS가 지원하는 날짜/시간 지정 형식 몇 가지는 다음과 같습니다.

```
"1971-04-30 04:35" (ISO format)
"30 Apr 1971 04:35" (Internet format)
"Tue Jan 25 08:45:45 UTC 2000"
"3/31/92 10:00:07 PST"
"January 23, 1987 10:05pm"
"22:00 GMT"
"1 month ago"
"2 hours ago"
"400000 seconds ago"
"last year"
"last Monday"
"yesterday"
"a fortnight ago"
```

1.33 CVS 서버에 어떤 module이 있는지 어떻게 알 수 있나요? ¶

[\[edit\]](#)

CVS에서 제공하는 명령어를 통해 정상적으로 알아낼 수 있는 방법은 **없습니다.** 😞 그렇지만 꼭 필요한 경우에 흔히 사용되는 방법이 몇 가지 있습니다.

1. 프로젝트 관리자에게 물어보세요 😊
2. 프로젝트 관리자가 수동으로 CVSROOT/modules 에 등록해 놓으면 다음 명령으로 module의 목록을 볼 수 있습니다.

```
$ cvs co -c
```

3. 잘 관리되는 프로젝트라면 일반적으로 cvsweb 이나 viewcvs 같은 웹 인터페이스를 제공할 것입니다. 그걸 이용하세요. 만약 없다면 서버 관리자에게 그런 프로그램을 설치해 달라고 요청하세요.
4. 위 방법이 모두 안된다면, 다음 스크립트를 써서 간접적으로 알아낼 수도 있습니다.

```
$ cvsIs.sh HEAD .
```

```
#!/bin/sh
```

```
# lists files and directories in the module(s) on the server without
# checking them out
```

```
# cvsIs BranchName modules...
# for main trunk use 'cvsIs HEAD modules...'
```

```
branch=$1; shift
```

```
cvs rdiff -s -D '01/01/1971' -r $branch "$@" 2>&1 &&
| sed -e 's/File.//' &&
    -e 's/is new: current revision./ (/ &&
    -e 's/(W{0-9}[W.0-9]*W)/(W1)/' &&
    -e 's/cvs server: Diffing/cvs server: Listing/'
```

1.34 CVSROOT/modules에 모듈 등록은 어떻게 하나요? ¶

[\[edit\]](#)

cvs co CVSROOT 하여서 modules 파일을 수정하신 후에 commit 하셔야 합니다. 이때 위와 같은 목적으로 활용하실 계획이라면 간단하게 다음과 같이 적으시면 됩니다.

```
module_A module_A
module_B module_B
```

위의 내용은 module_A라는 것이 repository root에 존재하는 경우 입니다.

1.35 CVSROOT/passwd에 사용자를 어떻게 추가하나요? ¶

[\[edit\]](#)

몇 가지 방법이 있지만, 가장 간단한 방법 가운데 하나는 apache 패키지에 들어 있는 **htpasswd** 명령을 이용하는 것입니다.

```
$ htpasswd -b passwd username password
```

1.36 익명 사용자는 어떻게 등록하나요? ¶

[\[edit\]](#)

```
$ cat $CVSROOT/CVSROOT/readers
cvs
cvsguest
```

1.37 CVS가 /root 아래 파일을 찾으려 듭니다! 이게 뭔가요? ¶

[\[edit\]](#)

pserver 설정을 해줄 때 아래와 같은 예러가 나는 경우가 있습니다.

```
cvs server: cannot open /root/.cvsignore: Permission denied
```

이럴 땐 inetd 나 xinetd 설정을 해줄 때 cvs에 넘겨줄 인수에 '-f' 옵션을 추가해주면 해결됩니다. xinetd 라면 /etc/xinetd.d/cvspserver 내용을 아래와 같이 해 주면 됩니다.

```
# default: on
# description: cvs pserver
service cvspserver
{
    disable          = no
    flags            = REUSE
    socket_type      = stream
    wait            = no
    user            = root
    server           = /usr/bin/cvs
    server_args      = -f --allow-root=/home/cvs pserver
    log_on_failure   += USERID
}
```

1.38 Repository를 다른 기계로 옮기고 싶습니다. ¶

[\[edit\]](#)

Q : 만일 A 서버에 있던 repository 를 B 서버로 옮기고 싶다면 그냥 디렉토리를 통채로 복사하면 되려나요? 그게 안 된다면 죄다 update 후 새로 import 를 하라는 얘기인데... _-;;

A : 네 그냥 옮기면 됩니다. 완벽한(?) 이전을 위해서라면 조금 신경써야할 것들이 더 있긴 하지만, 디렉토리를 통째 tar로 묶어서 새 기계에 풀어준 다음 permission 조절만 해줘도 웬만큼 됩니다.

1.39 CVS 서버에 있는 repository를 미러링할 수 없나요? ¶

[\[edit\]](#)

단순한 백업 이상을 의미하는 것이라면, CVS는 기본적으로 단일 repository를 바탕으로 설계되었기 때문에 자체적으로는 불가능합니다. 대규모 프로젝트 같은 경우에 전세계 곳곳에 미러를 하기 위해 CVSup 같은 보조도구를 이용하거나, 단순히 rsync 같은 미러링 툴을 이용하기도 합니다. 그러나 이 경우 하위 repository에 commit 한 내용을 다시 상위(upstream)로 merge 하는데 기술적인 어려움이 따릅니다. 그렇지만 경우에 따라 효과적인 개발 브랜치나 로컬 개발자 그룹 지원을 위해서는 꼭 필요한 기능이기에 때문에, CVS가 가지는 근본적인 제약을 극복하고자 새로 개발되는 프로그램이 있습니다. 만약

꼭 계층적 repository를 구축해야 하는 상황이라면, 다음 프로그램을 사용하는 것을 고려해 보세요.

- [arch](#)

최근에 [XFree86](#) 프로젝트를 arch를 이용해 fork한 사례가 있습니다. : <http://www.xouvert.org/>

1.40 module들을 공유할 수 있는 방법이 있나요? ¶

[[edit](#)]

Q : 예를 들어서 libx라는 모듈이 있고, proj_a와 proj_b가 이를 모두 사용합니다. 이때 이 두 프로젝트가 유사하여서 libx를 한쪽에서 수정하면 다른 한쪽에서도 영향을 받아야 하는 경우라면 어떻게 해야 할까요? 서버에서 symbolic link를 걸어주거나, 클라이언트측에서 proj_a, proj_b에 모두 libx를 checkout 해주면 되겠지만 다소 불편함이 없지 않아 있을것 같아서 여쭙 봅니다.

A : 가능합니다.

CVS에서는 프로젝트 관리를 위해 어떤 모듈을 checkout할 때 하위 모듈을 선별적으로 checkout하거나 가상 모듈을 만들어 실제 모듈을 여러개 동시에 checkout하는 것 등을 지원합니다. 기본적으로는 CVSROOT/modules 파일을 적절히 구성해서 프로젝트에 참여하는 개발자들이 필요한 부분을 잘 골라 받아갈 수 있도록 관리해줘야 합니다. 한가지 짚어두고 싶은 것은, CVS는 소스 버전 관리 프로그램이지 어떤 특정 프로젝트 구조를 정해둔게 아니기 때문에 실제 해결 방식은 진행하는 프로젝트 구성 및 정책에 따라 달라집니다.

간단하게 두가지 경우를 예로 들어보지요.

1. proj_a 와 proj_b, 그리고 libx를 더 큰 proj 으로 묶을 수 있는 경우

CVSROOT/modules 내용은

```
proj    proj    proj_a proj_b libx
proj_a  proj    proj_a libx
proj_b  proj    proj_b libx
libx    proj    libx
```

와 같이 되고, checkout을 하게 되면 proj 디렉토리가 먼저 생기고, 그 아래에 각 모듈이 생깁니다.

2. proj_a 와 proj_b 는 별개 프로젝트이고 libx 도 별개로 관리하는 경우

CVSROOT/modules 내용은

```
proj_a  -a proj_a libx
proj_b  -a proj_b libx
libx    libx
```

이렇게 되고, 예를 들어 cvs checkout proj_a 를 하게 되면 proj_a와 libx라는 모듈이 각각 생성됩니다. 이 경우 libx에 autotools(libtool)을 채용하든가 proj_a와 proj_b 빌드하는데 연결되도록 적절히 구성해 줘야 하겠죠.

두가지가 무슨 차이냐? 라고 혹시 생각하는 분이 있을지 모르겠는데, 전자에서는 최상위 디렉토리에 라이브러리를 포함한 빌드 시스템을 구축 관리할 수 있기 때문에 성격이 많이 달라집니다.

또, 경우에 따라

```
proj    proj
proj_b  proj    proj_a
proj_a  proj    proj_b
libx    libx
```

이런 식으로 구성하는 경우도 생기겠지요.

심볼릭 링크를 걸어주는 것도 편법으로 가능한 하지만, 모듈이나 라이브러리 관리라는 측면에서 본다면 그다지 바람직하지는 않습니다. 물론 불가피한 상황이 있을 수도 있으나 대개는 '라이브러리'라는 것 자체가 해당 코드를 별개로 독립시켜야할 만큼 여러 곳에서 쓰는 경우에 한해 만드는 것이므로 위에서 예를 든 두 가지 경우를 적절히 섞어서 구성하고, autotools 등을 잘 이용하면 해결할 수 있습니다.

([DeleteMe](#) 이 주제는 [CVS/FAQ](#)라는 범위를 벗어나는 것 같아 이정도만 적습니다.)

1.40.1 간단한 방법 ¶

[[edit](#)]

단순히 어떤 모듈을 가져올 때 특정 하부 모듈을 같이 가져오도록 하려면 다음과 같이 하면 됩니다.

```
proj_a proj_a &libx
```

```
proj_b proj_b &libx
libx libx
```

이렇게 하면 proj_a를 체크아웃할 때 proj_a 디렉토리 아래에 추가로 libx 디렉토리가 생깁니다.

1.41 Source file에 강제로 revision을 지정할 수 있나요? ¶

[\[edit\]](#)

Q : 예를 들어서 어떤 소스의 revision #가 1.12라고 할 때 어떤 계기로 인해서 이 번호의 제일 앞자리 숫자를 바꾸려고 하면 어떻게 하면 되나요 ?

A : 가능합니다.

```
cvs ci -m "revision set to 2.0" -r2.0 ohmysrc.c
```

해당 프로젝트/모듈 전체를 한번에 바꾸려면, checkout 받은 모듈 최상위 디렉토리에서

```
cvs ci -m "revision set to 3.1" -r3.1
```

와 같이 하면 됩니다.

그러나

가능하다고 해서 반드시 이렇게 하는게 좋다는 것은 아닙니다.

그러므로

Tag나 Branch를 써야하는 경우가 아닌지 다시 한번 생각해 보세요.

1.42 CVS도 VSS처럼 배타적 락(Exclusive Lock)을 걸어 놓고 사용할 수 있나요? ¶

```
cvs admin -l filename
```

을 이용하시면 됩니다. 이 경우 해당하는 사람이 lock을 제거하기 전까지는 commit이 불가능하게 됩니다. 하지만 이렇게 하는 것은 대부분 문서에서 그다지 권장하지 않더군요. cvs watch 이용하기를 권장하고 있습니다.

보통은 cvs edit를 써서 원하는 효과를 얻을 수 있습니다. cvs edit는 동시에 edit하는 것을 방지하지 않지만, cvs edit -c filename 을 모든 사람이 이용하게 된다면 그 순간에는 한 사람만이 작업을 할 수가 있게 됩니다. -c 옵션을 준 것을 **reserved edit**라고 하며, 이 경우에도 누구나 commit은 가능합니다.

1.43 Repository에 Attic 이란게 있던데 이건 뭔가요? ¶

[\[edit\]](#)

사용자가 지운(cvs rm) 파일을 보관해두는 곳입니다. Attic이 우리말로 하면 "다락"인데, 자주 쓰지는 않지만 버리긴 아깝고 혹시 필요하게 될지 모르는 물건을 저장한다는 점에서 보면 상당히 잘 지은 이름 같습니다.

사용자가 cvs rm을 해서 파일을 지우게 되면, 해당 파일들이 완전히 삭제되는게 아니라 Attic 아래에 옮겨집니다. 그리고 과거 버전을 checkout 하거나 할때 끄집어내서 쓰는 거지요.

1.44 Unicode file을 지원하지 못하는가요? ¶

[\[edit\]](#)

Q : 이러한 메시지를 만났습니다.

```
cvs import: Remote server does not support Unicode files. Checkin may be invalid.
```

여러개의 파일을 한꺼번에 import할때 나왔으며, 이를 다시 수정하여 ci할때에도 나타났습니다. 음.. 첫 번째 파일에서 이러한 문구가 나타났는데, 평범한 C++ code라서 다른 파일이 아닌가 합니다. 아울러 -ku라고 나타나는 메시지도 무엇인지 모르겠네요.

```
cvs commit: Remote server does not support Unicode files. Checkin may be invalid.
```

```
Checking in ***.cpp;
```

```
cvs server: internal error: unsupported substitution string -ku
```

```
/***.cpp,v <-- ***m.cpp
```

```
new revision: 1.2; previous revision: 1.1
```

```
done
```

해당 파일 안에 유니코드로 된 텍스트가 들어있나요? 혹시 RCS keyword substitution 으로 쓰이는 내용이 들어있나요?

1.45 이미 Text로 업로드 된 파일을 Binary로 바꿀 수 있는가요? ¶ [\[edit\]](#)

바이너리로 업로드하려면 -kb옵션을 사용합니다. 예를들어,

```
$ echo '$Id: CVS_2fFAQ,v 1.99 2005/08/19 01:21:43 kss Exp kss $' > kotest
$ cvs add -kb -m"A test file" kotest
$ cvs ci -m"First checkin; contains a keyword" kotest
```

입니다.

만약 실수로 -kb옵션을 빠뜨리고 (텍스트로)업로드 했을 경우, 바이너리로 복구하려면 cvs admin 옵션으로 아래의 예처럼 바꾸면 됩니다.

```
$ echo '$Id: CVS_2fFAQ,v 1.99 2005/08/19 01:21:43 kss Exp kss $' > kotest
$ cvs add -m"A test file" kotest
$ cvs ci -m"First checkin; contains a keyword" kotest
$ cvs admin -kb kotest
$ cvs update -A kotest
# For non-unix systems:
# Copy in a good copy of the file from outside CVS
$ cvs commit -m "make it binary" kotest
```

1.46 이미 Binary로 업로드 된 파일을 Text로 바꿀 수 있는가요? ¶ [\[edit\]](#)

한글이 들어있는 파일을 [WinCVS](#)를 통해서 add/import를 하게 되면 binary를 권장합니다. 이때, 실수로 Binary로 add/import하게 되면 merge기능을 지원하지 않아 불편하게 되지요. 이럴 때, 위의 Text-> Binary와는 반대의 작업이 필요하게 되는데, 위의 내용과 같고, 옵션만 -kb가 아닌 -kkv로 해 주면 됩니다.

```
$ cvs admin -kkv kotest
```

-- [DeleteMe](#) 이것으로 한참 고민하고 이것 저것 해 본 결과 알아 냈습니다만, 여기서 말하는 keyword라는 게 정확하게 의미하는 바를 모르겠습니다. 아시는 분은 설명 부탁드립니다. (-kkv의 help에는 Generate keywords using the default form.라고 되어 있습니다.) [barmi](#)

1.47 로그를 잘못 적었는데 고칠 수 없을까요? ¶ [\[edit\]](#)

```
$ cvs admin -m1.2:"new log" myprog.c
```

태그가 붙어 있는 경우라면

```
$ cvs admin -mV_1_0:"massive new log"
```

이렇게 한번에 바꾸는 것도 가능합니다.

1.48 Tag에 comment를 붙이는것은 가능한가요 ¶ [\[edit\]](#)

[AnswerMe](#) 질문이 조금 애매한것 같습니다. 제목만 보고는 어떤 상황인지 정확히 알 수 없어 답변이 두루뭉실합니다. 조금더 상황을 설명해 주시지요?

tag를 붙이면서 거기에 대한 로그를 어딘가에 기록해둘 수 없냐는 애긴가요? 그런거라면 대답은 불가능하다 입니다. CVS는 파일 별로 로그를 붙이게 돼 있는 RCS를 계승하기 때문에 각 checkin 별로 기록을 할 수 없다는 단점이 있습니다. 그래서 [Subversion](#) 같은 경우 아예 파일별 revision이 아니라 per-checkin revision을 모듈 단위로 붙여주는 방식을 채택했습니다.

만약 tag 자체에 대해 어딘가에 기록을 남기고 싶은 거라면 [ChangeLog](#) 나 README 등에 내용을 적어두는 것으로 대체할 수도 있습니다.

질문자입니다. 말씀해 주신 내용이 정확합니다. 제가 보기에요 file 단위로 log message를 남기는 것 같아서 여쭙 보았던 것입니다. 아직 익숙하지 못해서인지 좀 불편한 점으로 생각되어서 질문을 했던 것입니다. ☺

<http://www.cvstrac.org/> CVS mod인것 같은데 check-in 단위로 기록을 남길수 있습니다.

<http://www.sqlite.org> 에서 사용 하는걸 보면 괜찮아 보입니다.

1.49 Unknown command 라는 에러가 납니다. ¶ [\[edit\]](#)

Q: 접속시에

```
cvs [login aborted]: unrecognized auth response from 127.0.0.1: Unknown command: `/home/cvs'
```

라고 나오네요.. 강좌대로 했다고 생각했는데.. passwd file 문제인가요? redhat 9.0을 쓰고 있습니다.

A: -d 옵션 (CVSROOT) 줄 때 사용자@서버명:과 /디렉토리/경로 사이에 빈 칸이 들어가 있는 것 같습니다.

```
$ cvs -d :pserver:anonymous@anonymous.com:/cvsroot
```

이렇게 전부 붙여줘야 합니다.

1.50 항상 최신버전의 소스를 유지하고 싶어요 ¶

[\[edit\]](#)

항상 최신의 버전을 유지하지 않고 작업을 하니 Commit시 자주 Conflict가 발생합니다. [WinCVS](#)나 [TortoiseCVS](#)를 써 보았는데 최신 버전으로 유지해 주는 기능을 안 보이던데요. 덧붙여 update시 발생하는 conflict를 줄이고 싶습니다. update시 merge를 선택적으로 할 수는 없나요? 예를 들어서 '이 파일은 더 최근의 것이 존재합니다. merge할까요? (Y/n)' 이런식으로요.

* [WinCVS](#)에서 우선 update합니다. 그럼 merge나 conflict 되어 있을겁니다. 다시 update하면서 update settings중 Get the clean copy를 체크하시면 됩니다.
Clean copy를 받고 싶은게 아니라 local modify 파일은 update를 받고 싶지 않은 경우를 말씀 드렸답니다. 😊 그리고 update 명령을 자동으로 수행할 수 있는 방법이 있으면 좋겠다는거였습니다. 2가지가 상충된 질문을 드려서 헷갈리셨나 봅니다. - 감사합니다.
그런 경우 Update query 해서 저장소에 변경된 파일을 확인한 후 그 파일들만 선택해서 업데이트 하시면 됩니다. 스크립트 하나 만들어 보시는 것도 좋을듯 합니다. 😊

1.51 이전버전으로 되돌리고 싶어요(Rollback) ¶

[\[edit\]](#)

| <http://bbs.kldp.org/viewtopic.php?t=28936> |

1.52 CVS를 쓸까요 Subversion을 쓸까요? ¶

[\[edit\]](#)

Q : 이제서야 CVS라는 것에 대해 알게 되어 배워볼까 했더니 CVS의 불편한 점을 개선한 Subversion이 나왔다고 하네요. 처음 배우는 사람은 바로 Subversion을 익혀야 하나요? 아니면 CVS 를 먼저 익혀야 하나요?

A : 대부분의 오픈소스가 CVS로 진행 되니 CVS를 먼저 익히는것이 좋겠죠. Subversion이 안정적이라고 하긴 하나 공식적인 stable 버전은 언제쯤 나올지는... 요즘 한달에 두번씩 버전이 올라가는데 업데이트 해주기 좀 귀찮더군요. CVS에 불만이 생기면 그때 슬슬 바꾸시면 됩니다.

A : [SubVersion](#)은 CVS의 불편한 점을 개선했다기 보다는 완전히 다른 프로그램으로 보는게 맞습니다. 상호 호환성도 없고, 단지 [SubVersion](#)에서 기존에 CVS로 운영되던 프로젝트를 import할 수 있게 해놨을 따름입니다. revision/version 관리 모델 자체도 전혀 다릅니다. 시기상 나중에 만들어졌기 때문에 CVS가 가지는 단점들을 상당 부분 개선한 것은 사실입니다만, 개발 모델 자체에 대한 접근 방식이랄까 철학이랄까 그런게 다른 부분이 있어 보여서 제 경우엔 조금 관망하는 중입니다. 버전 컨트롤 프로그램을 전혀 써보지 않은 사람 입장에서 어느 쪽이 더 익히기 쉽다든가 하는 건 잘 모르겠군요.

[SubVersion](#)도 CVS를 쓰던 사람은 쉽게 옮겨갈 수 있다고 얘기합니다만 그거랑은 별개라서 말이죠.

--[verotas](#)

2 help 사용법 ¶

[\[edit\]](#)

버전 확인

```
$ cvs --version
Concurrent Versions System (CVS) 1.11.2 (client/server)
```

그냥 cvs만 치면 help 사용법이 자세히 나온다.

```
$ cvs
Usage: cvs [cvs-options] command [command-options-and-arguments]
  where cvs-options are -q, -n, etc.
    (specify --help-options for a list of options)
  where command is add, admin, etc.
    (specify --help-commands for a list of commands
    or --help-synonyms for a list of command synonyms)
  where command-options-and-arguments depend on the specific command
```


(specify -H followed by a command name for command-specific help)
Specify --help to receive this message

The Concurrent Versions System (CVS) is a tool for version control.
For CVS updates and additional information, see
the CVS home page at <http://www.cvshome.org/> or
Pascal Molli's CVS site at <http://www.loria.fr/~molli/cvs-index.html>

특정 명령어 옵션을 자세히 보려면

```
$ cvs -H update
Usage: cvs update [-APCdfIRp] [-k kopt] [-r rev] [-D date] [-j rev]
        [-l ign] [-W spec] [files...]
.....
```

사용할 수 있는 명령어 생략형은 다음과 같다.

```
$ cvs --help-synonyms
CVS command synonyms are:
    add          ad new
    admin        adm rcs
    annotate      ann
    checkout     co get
    commit       ci com
    diff         di dif
    export       exp ex
    history      hi his
    import       im imp
    log          lo
    login        logon lgn
    rannotate    rann ra
    rdiff        patch pa
    release      re rel
    remove       rm delete
    rlog         rl
    rtag         rt rfreeze
    status       st stat
    tag          ta freeze
    update       up upd
    version      ve ver
```

(Specify the --help option for a list of other help options)

3 용어 ¶

[\[edit\]](#)

repository working file working directory (or area) checkout commit (checkin) RCS file modules
revision tag branch "the trunk" HEAD / BASE merge conflict

4 문서를 만들면서 ¶

[\[edit\]](#)

개인적으로, 몇 년 전 CVS를 처음 배우면서 제일 어려웠던게 간단한 소개 문서는 한글로 된게 있었지만 조금만 더 상세한 걸 찾아보려면 제대로 된 (영어든 한글이든) HOWTO 같은게 있지도 않았고, (지금도 있는지는 모르겠지만) Faq-O-Matic 으로 된 영어 FAQ 사이트가 레퍼런스로 쓸만한 유일한 곳이었습니다. 그래서 그때부터도 처음 오픈소스 세상을 접하는 개발자에게 CVS라는 장벽을 뛰어 넘는데 보탬이 되는 문서 같은게 꼭 필요하다고 느끼고 있었습니다.

그동안 CVS를 다룬 책도 출간되었고 CVS 다음 세대가 될만한 다른 프로그램들도 나왔고 해서 하려고 만 든다면 상대적으로 자료 찾거나 배우기가 더 쉬워진 면도 분명 있습니다. 그렇지만 본래 좋은 HOWTO 들이 그러하듯이 처음 시작하는 사람이 딱 그 문서 하나만 짚어서 옆에 두고 손으로 짚어가며 따라하면 일단 "시작은 어떻게든 해볼 수 있는 글이 아쉬웠고, 글 서두에 밝혔듯이 현재 존재하는 한글로 된 CVS 관련 문서들은 담고 있는 정보가 가지는 유용성에 비해 집약도(?)가 떨어지고, 사용자에게 필요한 내용과 관리자에게 필요한 내용이 잘 구분이 되어 있지 않아 결과적으로 처음 접하는 사람에게 문턱을 낮추는 역할에 충실하지 못한 부분이 있다고 생각합니다.

장기적으로는 이런 것들을 해결할 수 있는 좋은 글이 나와야 하겠지만, 아쉬운 대로 "KLDP에 있는 문서를 보고 따라하다 보니 이런 문제에서 걸리는데 어떻게 하면 되는 거냐" 하는 질문에 대한 답 정도라도 생각나는 대로 정리해 보자 라고 생각해서 시작하게 됐습니다. 앞으로도 생각나는 대로 항목들 추가해 나가면서 정리해 나갈 생각입니다. --[verotas](#)

Thanks to [권순선](#), [종아](#), [voxel](#), [Irlz](#), [Advanced](#)

5 관련 자료 ¶

[[edit](#)]

- [The CVS Manual](#) 가장 정식적인 매뉴얼. 이 내용 그대로 인쇄된 책도 팔고 있죠. 배경지식 없이 처음부터 읽어나가는 조금 부담스럽기도 합니다.
- [CVS Best Practices \(TLDP\)](#) CVS 관련 (영문) 문서 가운데 가장 압축적으로 잘 쓴 문서 아닌가 싶습니다. 한번 훑어만 봐도 CVS 사용에 대해 한단계 업그레이드할 수 있는 문서. 중급자용 ^^;



- [The CVS Book](#)
 - [HomePage](#)에서 html / pdf 형식 온라인 문서를 제공합니다.
 - 2003년 8월에 3판이 나왔습니다.
 - 특히 tag / branch 가 뭐고 어디에 어떻게 쓰는 물건인지 궁금한 분들은 이 책을 차근차근 읽어보세요.
 - 자매품 [The Subversion Book](#), 한글번역문서 [Subversion_Book](#)도 있습니다. 2003년 하반기에 출간될 예정이라는군요.
- 한글로 된 다른 문서 ([hey](#)님 위키에서)
 - [CVS 이야기\(KLDP\)](#)
 - [CVS 사용\(KLDP\)](#)
 - [CVS 소개](#)
 - [CVS 관리](#)
 - [CVS에서 유용하게 사용되는 변화 추적\(Change-tracking\)](#)
 - [CVS User Guide](#)
 - [HowToUseCvs](#)
 - cvs 서버 세팅하기 : <http://people.kldp.org/~eunjea/hcol-archive/2002/apps/2002-Feb/0013.html>
- 관련 링크
 - [CVS HomePage](#)
 - [CVS FAQ-O-Matic](#)
 - [CVS Quick Reference Card \(PDF\)](#)

[CategorySoftwareTool](#)

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

[W3C](#) [XHTML 1.0](#) [W3C](#) [CSS](#) [Powered by](#) [Pycasa](#)

Last modified: 2005-08-19 10:21:44, Processing time: 0.5108 sec

[회사일정관리](#)

Google에서 제공하는 전사용 무료 공유 캘린더 도구

[웹 사진 앨범](#)

Picasa 웹앨범으로 간단하게 온라인으로 사진 게시 및 공유하기